

Name : Abdullah Saood

Natural Language Processing

Transformer-Based Sentiment Analysis with Explain ability

Output Report:

Code:

```
!pip install transformers datasets torch scikit-learn shap lime seaborn -q
!pip install --upgrade transformers -q
import transformers
print(transformers.__version__)
```

Output:

```
10.6/10.6 MB 77.2 MB/s eta 0:00:00:00:01:01
661.5/661.5 kB 25.1 MB/s eta 0:00:00
4.5/4.5 MB 96.0 MB/s eta 0:00:00:00:01
5.8.0
```

Code:

```
import torch
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from datasets import load_dataset
from transformers import AutoTokenizer, AutoModelForSequenceClassification
from transformers import Trainer, TrainingArguments
from sklearn.metrics import accuracy_score, precision_recall_fscore_support

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print("Using device:", device)
```

Output:

```
Using device: cuda
```

Code:

```
dataset = load_dataset("fancyzhx/amazon_polarity")

train_data = dataset["train"].shuffle(seed=42).select(range(15000))
test_data = dataset["test"].shuffle(seed=42).select(range(3000))
```

Output:

README.md:  6.81k/? [00:00<00:00, 617kB/s]

Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.

amazon_polarity/train-00000-of- 260M/260M [00:03<00:00, 112MB/s]
00004.par(...): 100%

amazon_polarity/train-00001-of- 258M/258M [00:03<00:00, 90.2MB/s]
00004.par(...): 100%

amazon_polarity/train-00002-of- 255M/255M [00:02<00:00, 87.5MB/s]
00004.par(...): 100%

amazon_polarity/train-00003-of- 254M/254M [00:02<00:00, 150MB/s]
00004.par(...): 100%

amazon_polarity/test-00000-of- 117M/117M [00:01<00:00, 94.9MB/s]
00001.parq(...): 100%

Generating train split: 100%  3600000/3600000 [00:06<00:00, 568219.13 examples/s]

Generating test split: 100%  400000/400000 [00:00<00:00, 583725.89 examples/s]

Code:

```
tokenizer = AutoTokenizer.from_pretrained("bert-base-uncased")

def tokenize(example):
    return tokenizer(example["content"], truncation=True, padding="max_length", max_length=512)

train_data = train_data.map(tokenize, batched=True)
test_data = test_data.map(tokenize, batched=True)

train_data.set_format(type="torch", columns=["input_ids", "attention_mask", "label"])

test_data.set_format(
    type="torch",
    columns=["input_ids", "attention_mask", "label"],
    output_all_columns=True
)
```

Output:

config.json: 100%  570/570 [00:00<00:00, 53.6kB/s]
tokenizer_config.json: 100%  48.0/48.0 [00:00<00:00, 6.58kB/s]
vocab.txt:  232k/? [00:00<00:00, 8.39MB/s]
tokenizer.json:  466k/? [00:00<00:00, 11.4MB/s]
Map: 100%  15000/15000 [00:02<00:00, 5534.12 examples/s]
Map: 100%  3000/3000 [00:00<00:00, 3502.24 examples/s]

Code:

```
model = AutoModelForSequenceClassification.from_pretrained(  
    "bert-base-uncased",  
    num_labels=2,  
    output_attentions=False,  
    ignore_mismatched_sizes=True  
)  
  
model.to(device)
```

Output:

model.safetensors: 100%  440M/440M [00:02<00:00, 314MB/s]
Loading weights: 100%  199/199 [00:00<00:00, 4142.16it/s]

Code:

```
def compute_metrics(eval_pred):  
    logits, labels = eval_pred  
    preds = np.argmax(logits, axis=1)  
  
    precision, recall, f1, _ = precision_recall_fscore_support(labels, preds, average='bin'  
    acc = accuracy_score(labels, preds)  
  
    return {  
        "accuracy": acc,  
        "f1": f1,  
        "precision": precision,  
        "recall": recall  
    }
```

```

training_args = TrainingArguments(
    output_dir="./results",
    eval_strategy="epoch",
    save_strategy="epoch",
    per_device_train_batch_size=4,
    per_device_eval_batch_size=4,
    num_train_epochs=2,
    logging_dir="./logs",
    fp16=True # for GPU acceleration
)

trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=train_data,
    eval_dataset=test_data,
    compute_metrics=compute_metrics
)

trainer.train()

```

Output:

```

return super().apply(*args, **kwargs) # type: ignore[misc]
[3750/3750 11:00, Epoch 2/2]

```

Epoch	Training Loss	Validation Loss	Accuracy	F1	Precision	Recall
1	0.705197	0.529186	0.894667	0.896053	0.897233	0.894875
2	0.450048	1.008531	0.897667	0.899902	0.893204	0.906702

```

Writing model shards: 100% [100%] 1/1 [00:00<00:00, 1.16it/s]
/usr/local/lib/python3.12/dist-packages/torch/autograd/function.py:583: UserWarning: Was asked to
gather along dimension 0, but all input tensors were scalars; will instead unsqueeze and return a
vector.
return super().apply(*args, **kwargs) # type: ignore[misc]
Writing model shards: 100% [100%] 1/1 [00:00<00:00, 1.24it/s]
7]: TrainOutput(global_step=3750, training_loss=0.610125146484375, metrics={'train_runtime': 662.4501,
'train_samples_per_second': 45.286, 'train_steps_per_second': 5.661, 'total_flos': 98666645760000
0.0, 'train_loss': 0.610125146484375, 'epoch': 2.0})

```

Code:

```
results = trainer.evaluate()
print(results)
```

Output:

```
100%|#####| [375/375 00:24]
```

Training Loss	Validation Loss	Epoch	Accuracy	F1	Precision	Recall
0.450048	1.008531	2	0.897667	0.899902	0.893204	0.906702

```
{'eval_loss': 1.008531093597412, 'eval_accuracy': 0.8976666666666666, 'eval_f1': 0.8999021845451581, 'eval_precision': 0.8932038834951457, 'eval_recall': 0.9067017082785808}
```

Code:

```
from transformers import AutoModelForSequenceClassification

model_attn = AutoModelForSequenceClassification.from_pretrained(
    "bert-base-uncased",
    num_labels=2,
    attn_implementation="eager",
    output_attentions=True
)

model_attn.load_state_dict(model.state_dict())

model_attn.to(device)
model_attn.eval()
```

Output:

```
Loading weights: 100% ##### 199/199 [00:00<00:00, 2394.98it/s]
```

Code:

```
model_attn.load_state_dict(model.state_dict())
```

```
<All keys matched successfully>
```

Code:

```
def visualize_attention(text, layer=1, head=0):
    inputs = tokenizer(text, return_tensors="pt").to(device)

    with torch.no_grad():
        outputs = model_attn(
            **inputs,
            output_attentions=True,
            attn_implementation="eager"  # 🔥 FORCE FIX HERE
        )

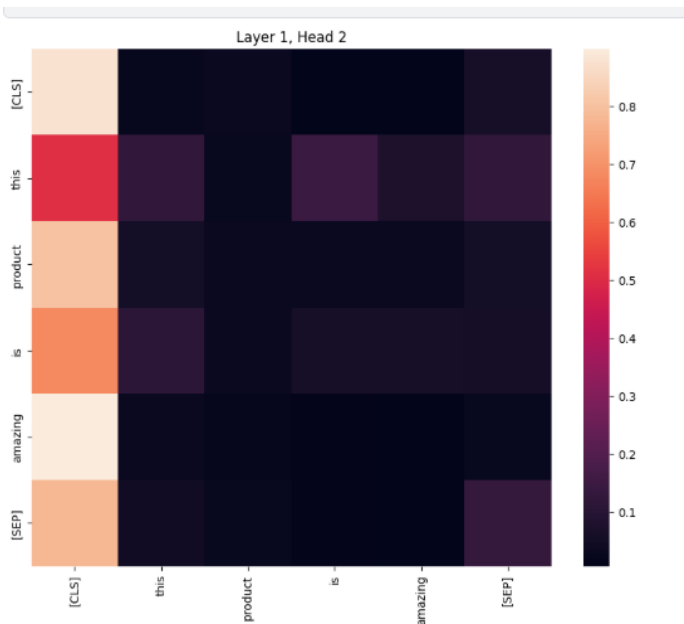
    attentions = outputs.attentions[layer][0][head].cpu().numpy()
    tokens = tokenizer.convert_ids_to_tokens(inputs["input_ids"][0])

    import matplotlib.pyplot as plt
    import seaborn as sns

    plt.figure(figsize=(10,8))
    sns.heatmap(attentions, xticklabels=tokens, yticklabels=tokens)
    plt.title(f"Layer {layer}, Head {head}")
    plt.xticks(rotation=90)
    plt.show()

visualize_attention("This product is amazing", 1, 2)
```

Output:



Code:

```
# SHAP FIXED VERSION

import shap
import numpy as np

def predict(texts):
    # convert SHAP inputs properly
    if isinstance(texts, np.ndarray):
        texts = texts.tolist()

    texts = [str(t) for t in texts] # ensure all are strings

    inputs = tokenizer(
        texts,
        return_tensors="pt",
        padding=True,
        truncation=True,
        max_length=64
    ).to(device)

    with torch.no_grad():
        outputs = model(**inputs)

    return outputs.logits.cpu().numpy()

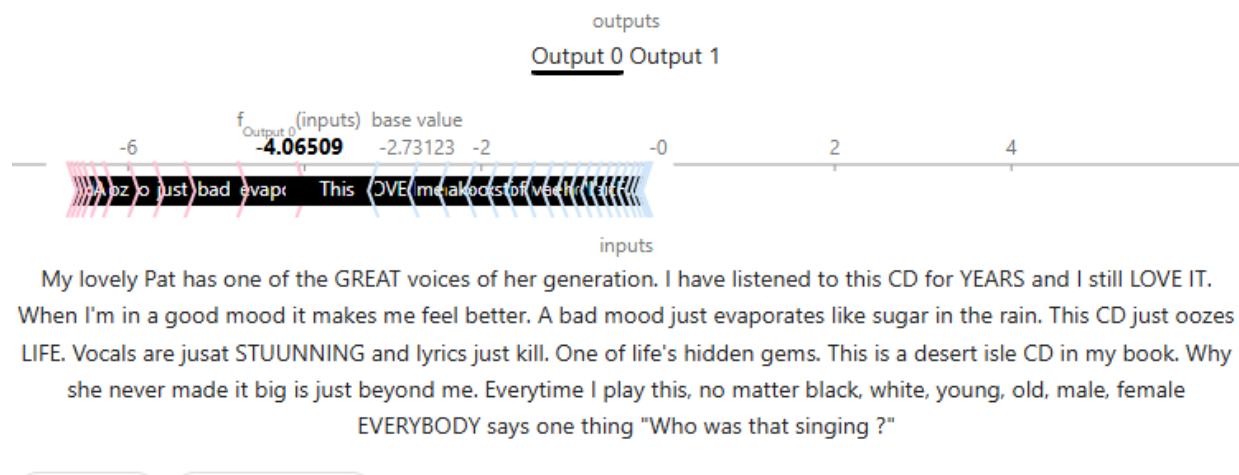
masker = shap.maskers.Text(tokenizer)
explainer = shap.Explainer(predict, masker)

sample_texts = [dataset["test"][1]["content"] for i in range(20)]

shap_values = explainer(sample_texts[:3])

shap.plots.text(shap_values[0])
```

Output:



Code:

```
from line.line_text import LineTextExplainer
import torch

explainer_line = LineTextExplainer(class_names=["Negative", "Positive"])

# clear GPU
torch.cuda.empty_cache()

def predict_proba(texts):
    model_cpu = model.to("cpu") # 🚀 move to CPU

    inputs = tokenizer(
        texts,
        return_tensors="pt",
        padding=True,
        truncation=True,
        max_length=64
    )

    with torch.no_grad():
        outputs = model_cpu(**inputs)

    probs = torch.nn.functional.softmax(outputs.logits, dim=1)

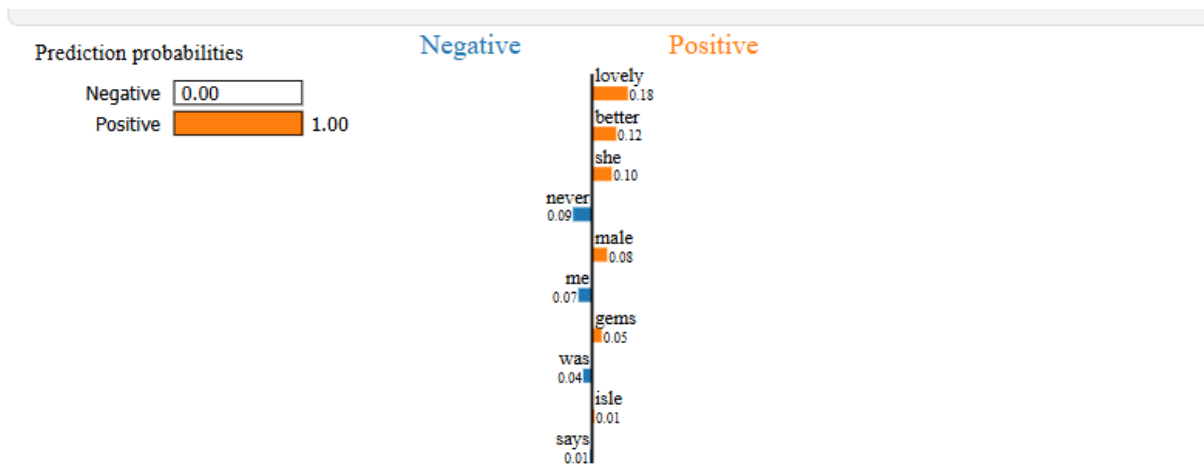
    return probs.numpy()

text = dataset["test"][0]["content"]

exp = explainer_line.explain_instance(
    text,
    predict_proba,
    num_features=10,
    num_samples=100
)

exp.show_in_notebook()
```

Output:



Text with highlighted words

My **lovely** Pat has one of the GREAT voices of her generation. I have listened to this CD for YEARS and I still LOVE IT. When I'm in a good mood it makes **me** feel **better**. A bad mood just evaporates like sugar in the rain. This CD just oozes LIFE. Vocals are jusat STUUNNING and lyrics just kill. One of life's hidden **gems**. This is a desert isle CD in my book. Why **she never** made it big is just beyond **me**. Everytime I play this, no matter black, white, young, old, **male**, female EVERYBODY says one thing "Who was **was** that singing ?"

Code:

```
] : model.to(device)

predictions = trainer.predict(test_data)

y_pred = np.argmax(predictions.predictions, axis=1)
y_true = predictions.label_ids

errors = []

for i in range(len(y_pred)):
    if y_pred[i] != y_true[i]:
        errors.append(dataset["test"][i]["content"])

print("Sample Errors:\n")
for i in range(5):
    print(errors[i], "\n")
```

Output:

Sample Errors:

My lovely Pat has one of the GREAT voices of her generation. I have listened to this CD for YEARS and I still LOVE IT. When I'm in a good mood it makes me feel better. A bad mood just evaporates like sugar in the rain. This CD just oozes LIFE. Vocals are just STUNNING and lyrics just kill. One of life's hidden gems. This is a desert isle CD in my book. Why she never made it big is just beyond me. Everytime I play this, no matter black, white, young, old, male, female EVERYBODY says one thing "who was that singing ?"

Exotic tales of the Orient from the 1930's. "Dr Shen Fu", a Weird Tales magazine reprint, is about the elixir of life that grants immortality at a price. If you're tired of modern authors who all sound alike, this is the antidote for you. Owen's palette is loaded with splashes of Chinese and Japanese colours. Marvelous.

I believe I will be very happy with the camcorder. Its small,light and easy to handle.The LCD is clear and bright.Ease of operation.

Very disappointed in this product. It worked perfectly for exactly three days and could not be resuscitated. It was very inexpensive so I did not want to pay half again the price to ship it back for an exchange, so the company would do nothing when they sent me an inquiry as to product satisfaction.

I had never read this author, but knew of many who did. When I read that this book was being made into a movie, it seemed a good opportunity to read her, before seeing the screen mess up her work. Now, I pray that the screen does so.I was extremely disappointed with this work. It is unrealistic, fails to capture the reader, and at points when I should have been completely engrossed, left me struggling with whether I wanted to pick it up. I hope she has written better than this, or else, there are a lot of people who have not bothered reading some of the strong authors available.Her characters are not sympathetic, they do not draw the reader's attention, and they are dull in overall scope. I have too many other good books to read before I pick up another book from this author.